

Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Algorytmy i Złożoność Obliczeniowa

Projekt 1 — Analiza algorytmów sortowania

Ocena docelowa: 5.0

Autor: Ivan Taborskykh 288347

Prowadzący: mgr inż. Damian Mroziński

Data: 11.05.2026

Spis treści

1	Wstęp	2
2	Opis zaimplementowanych algorytmów	2
2.1	Bucket Sort	2
2.2	Quick Sort	2
2.3	Shell Sort	3
3	Opis struktur danych	3
4	Metodologia badań	3
5	Wyniki badań	4
5.1	Badanie α — wpływ parametrów algorytmu	4
5.1.1	QuickSort: strategię wyboru pivotu	4
5.1.2	ShellSort: formuły sekwencji odstępów	5
5.2	Badanie A — wpływ rozmiaru zbioru	5
5.3	Badanie B — wpływ rozkładu danych wejściowych	6
5.4	Badanie C — wpływ typu danych	7
5.5	Badanie Ω — wpływ struktury danych	8
6	Wnioski	9

1 Wstęp

Celem projektu była implementacja trzech algorytmów sortowania — *Bucket Sort*, *Quick Sort* oraz *Shell Sort* — i zbadanie ich wydajności w zależności od kilku czynników: parametrów wewnętrznych algorytmu, rozmiaru danych, rozkładu danych wejściowych, typu przechowywanych elementów oraz zastosowanej struktury danych.

Wszystkie algorytmy zaimplementowano w C++ wersji 17 z użyciem szablonów (`template`), co umożliwiło jednolite testowanie na tablicach dynamicznych, listach jednokierunkowych, listach dwukierunkowych, stosach oraz drzewach BST. Czas sortowania mierzono wyłącznie dla operacji sortowania (bez wczytywania danych i bez alokacji pamięci) przy użyciu `std::chrono::high_resolution_clock::now()`, a wynik rzutowano na `std::chrono::microseconds`. Każdy wariant powtórzono 67 razy, a jako wynik przyjmowano średnią arytmetyczną.

2 Opis zaimplementowanych algorytmów

2.1 Bucket Sort

Bucket Sort (sortowanie przez podział na wiadra) działa w trzech fazach. Najpierw wyznacza zakres wartości w tablicy wejściowej ($[\min, \max]$) i tworzy $k = \lfloor \sqrt{n} \rfloor$ pustych wiadr, gdzie n to liczba elementów. Następnie każdy element jest przypisywany do wiadra proporcjonalnie do jego wartości w tym zakresie. Na koniec każde wiadro jest sortowane osobno algorytmem wstawiania (*insertion sort*), a wyniki scalane w jedną posortowaną tablicę.

Złożoność całkowita wynosi $O(n + n^2/k)$: dystrybucja elementów to $O(n)$, a przy jednorodnym rozkładzie danych każde wiadro zawiera n/k elementów, więc łączny koszt insertion sort wynosi $k \cdot O((n/k)^2) = O(n^2/k)$. Przy zastosowanym wyborze $k = \lfloor \sqrt{n} \rfloor$ upraszcza się to do $O(n\sqrt{n}) = O(n^{3/2})$.

Algorytm obsługuje wyłącznie typy liczbowe (`int`, `float`, `double`, `unsigned long`). W przypadku łańcuchów znaków sortowanie zamiennie wykonuje Shell Sort.

2.2 Quick Sort

Quick Sort (sortowanie szybkie) w zaimplementowanej wersji używa dwupunktowego schematu partycji: *pivot* jest przestawiany na koniec podciągu, po czym dwa wskaźniki zbiegają się ku sobie, zamieniając elementy poza miejscem. Aby uniknąć przepełnienia stosu wywołań przy głębokiej rekurencji (np. dla danych posortowanych), zastosowano implementację iteracyjną z jawnym stosem.

Dostępne są trzy strategie wyboru elementu osiowego (*pivot*):

- **Losowy** (-p 0) — *pivot* wylosowany z aktualnego podciągu;
- **Lewy** (-p 1) — pierwszy element podciągu;
- **Prawy** (-p 2) — ostatni element podciągu;
- **Środkowy** (-p 3) — element ze środka podciągu.

Złożoność oczekiwana wynosi $O(n \log n)$, natomiast pesymistyczna $O(n^2)$ — ta ostatnia pojawia się gdy pivot jest zawsze elementem skrajnym na danych posortowanych lub odwrotnie posortowanych.

2.3 Shell Sort

Shell Sort (sortowanie Shella) to uogólnienie sortowania przez wstawianie, gdzie elementy porównuje się i przestawia w dużych odstępach, stopniowo zmniejszając odstęp do 1. Dzięki temu elementy dalekie od swoich docelowych pozycji są szybko przesuwane w odpowiednim kierunku.

Zaimplementowano dwie sekwencje odstępów:

- **Formuła Shella** (-e 0) — $h = \lfloor n/2^k \rfloor$; złożoność $O(n^2)$ w pesymistycznym przypadku;
- **Formuła Knutha** (-e 1) — $h_1 = 1, h_{k+1} = 3h_k + 1$; złożoność $O(n^{3/2})$.

3 Opis struktur danych

Wszystkie zaimplementowane struktury danych są dynamiczne i oparte na szablonach, bez użycia standardowych kontenerów STL. Każda struktura udostępnia metody `toArray()` i `fromArray()`, co pozwala na jednolite sortowanie: dane są konwertowane do tablicy, sortowane, a następnie przepisywane z powrotem.

Tablica dynamiczna Ciągły blok pamięci; dostęp losowy $O(1)$.

Lista jednokierunkowa Węzły z polem `next`; dostęp sekwencyjny, bez dostępu losowego.

Lista dwukierunkowa Węzły z polami `next` i `prev`; umożliwia iterację w obu kierunkach.

Stos Struktura LIFO oparta na liście jednokierunkowej; operacje `push/pop`.

Drzewo BST Drzewo przeszukiwania binarnego; wstawianie elementów buduje drzewo, a przejście *in-order* zwraca je posortowane. Czas sortowania obejmuje budowę drzewa i jego przejście.

4 Metodologia badań

Środowisko testowe: MacBook Air (Apple M3), system macOS 14.6 (Sonoma), kompilator Apple Clang 16.0.0 (arm64). Program kompilowano z flagami `-O2 -Wall -Wextra -Werror`.

Dla każdego wariantu testu wykonywano 67 niezależnych pomiarów. Dane generowano od nowa przed każdym pomiarem. Mierzono wyłącznie czas operacji sortowania. Po każdym pomiarze weryfikowano poprawność sortowania; błędne pomiary były pomijane, jednak w żadnym przypadku nie wystąpiły błędy sortowania. Wyniki agregowano jako: średnia, minimum i maksimum czasu wykonania.

Rozmiary zbiorów w badaniu A: 10 000, 25 000, 50 000 i 100 000 elementów. Badania B, C, Alpha i Omega przeprowadzono dla $n = 25\,000$ elementów (wartość środkowa z badania A).

5 Wyniki badań

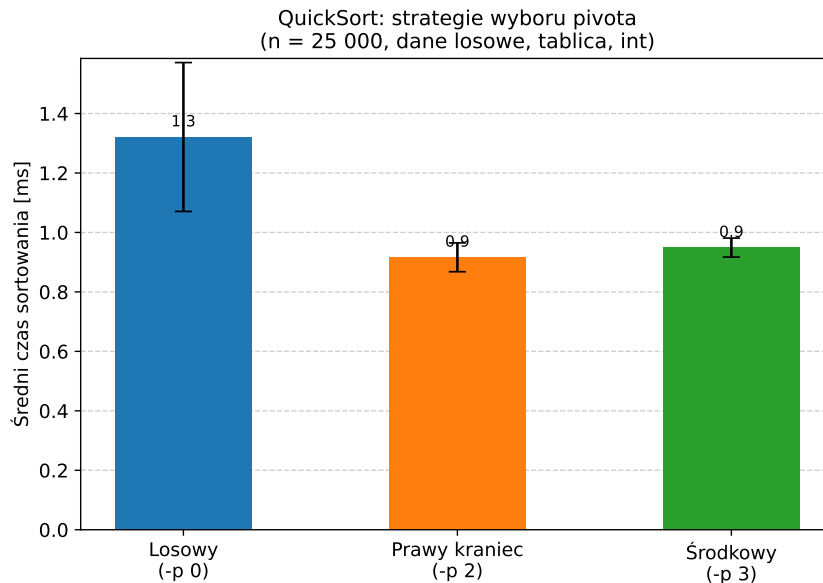
5.1 Badanie α — wpływ parametrów algorytmu

5.1.1 QuickSort: strategie wyboru pivotu

W tym badaniu porównano trzy strategie wyboru elementu osiowego dla zbioru 25 000 losowych liczb całkowitych na tablicy. Wyniki zestawiono w tabeli 1.

Tabela 1: Czas sortowania QuickSort w zależności od strategii pivotu ($n = 25\,000$, dane losowe, typ int).

Strategia pivotu	Średnia [μs]	Min [μs]	Max [μs]
Losowy (-p 0)	1 321	1 076	2 092
Prawy kraniec (-p 2)	916	857	1 084
Środkowy (-p 3)	949	913	1 045



Rysunek 1: Średni czas sortowania QuickSort dla trzech strategii wyboru pivotu.

Pivot losowy okazał się ok. 44% wolniejszy od pivotu prawego i ok. 39% wolniejszy od środkowego. Wynika to z narzutu wywołania `std::uniform_int_distribution` przy każdym kroku podziału — przy $n = 25\,000$ takich wywołań jest rząd n , co daje mierzalny narzut łączny.

Istotna jest też wyższa zmienność: odchylenie standardowe dla pivotu losowego wyniosło $\approx 250\ \mu s$, podczas gdy dla pivotu prawego i środkowego odpowiednio ≈ 49 i $\approx 32\ \mu s$. Wysoka wariancja wynika z tego, że losowy pivot może czasem trafić blisko krańca podciągu i wygenerować niezbalansowany podział — choć przy danych losowych zdarza się to rzadko.

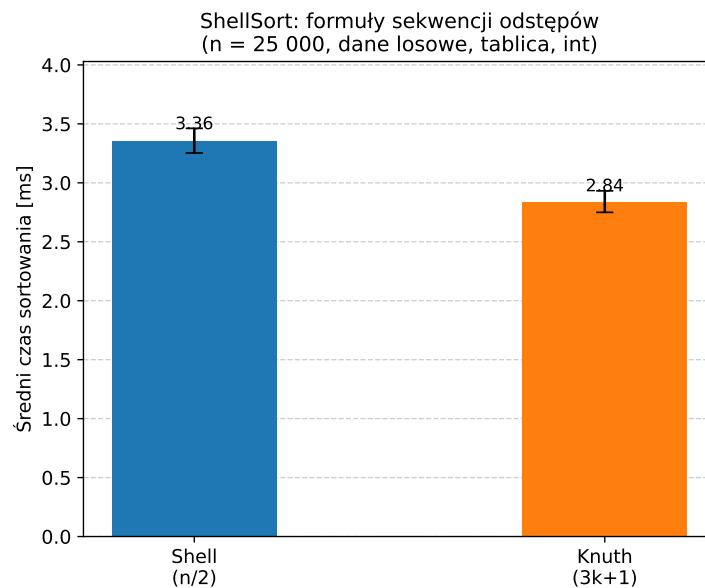
Na danych losowych pivot prawy i środkowy osiągają zbliżone czasy i są bardziej przewidywalne, więc losowość nie przynosi tu żadnej korzyści praktycznej.

5.1.2 ShellSort: formuły sekwencji odstępów

Tabela 2 i rysunek 2 przedstawiają porównanie dwóch sekwencji odstępów Shell Sorta.

Tabela 2: Czas sortowania ShellSort w zależności od formuły odstępów ($n = 25\,000$, dane losowe, typ int).

Formuła	Średnia [μs]	Min [μs]	Max [μs]
Shell ($n/2$)	3 357	3 129	3 596
Knuth ($3k+1$)	2 840	2 642	3 190



Rysunek 2: Średni czas sortowania ShellSort dla dwóch sekwencji odstępów.

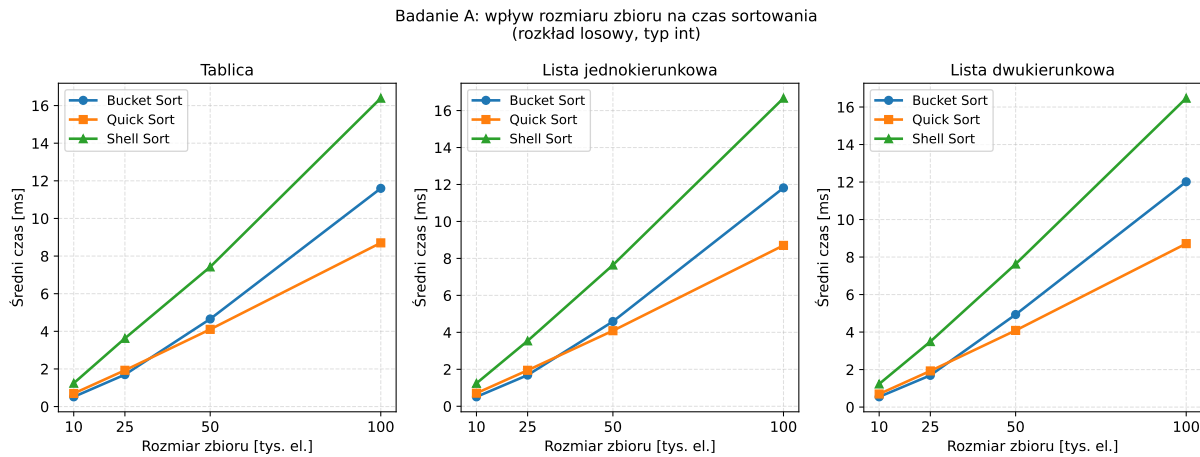
Formuła Knutha okazuje się o ok. 15% szybsza od oryginalnej formuły Shella. Wynika to z lepszych własności sekwencji $1, 4, 13, 40, \dots$: liczba faz jest zbliżona, ale każda faza lepiej redukuje liczbę inwersji, ponieważ wartości odstępów są dobrane tak, aby nie były wielokrotnościami poprzednich — co minimalizuje powtarzanie podobnych porównań.

5.2 Badanie A — wpływ rozmiaru zbioru

Badanie A mierzyło czas sortowania dla czterech rozmiarów zbiorów i wszystkich dziewięciu kombinacji algorytm $\times$ struktura. Dane były losowe, typ int. Wyniki dla tablic i list pokazuje rysunek 3, a szczegółowe wartości tabela 3.

Tabela 3: Średni czas sortowania [μs] w zależności od algorytmu, struktury i rozmiaru zbioru.

Algorytm	Struktura	$n = 10\,000$	$n = 25\,000$	$n = 50\,000$	$n = 100\,000$
Bucket Sort	Tablica	516	1 697	4 656	11 600
	Lista jednokierunkowa	509	1 685	4 587	11 816
	Lista dwukierunkowa	545	1 691	4 935	12 015
Quick Sort	Tablica	696	1 928	4 104	8 702
	Lista jednokierunkowa	709	1 947	4 088	8 704
	Lista dwukierunkowa	697	1 927	4 087	8 716
Shell Sort	Tablica	1 242	3 627	7 424	16 393
	Lista jednokierunkowa	1 242	3 539	7 642	16 670
	Lista dwukierunkowa	1 234	3 492	7 627	16 471



Rysunek 3: Średni czas sortowania w zależności od rozmiaru zbioru dla trzech algorytmów i trzech struktur liniowych.

Wyniki ujawniają kilka ciekawych zależności. Przy $n = 10\,000$ Bucket Sort jest najszybszy ($\approx 510\ \mu\text{s}$), natomiast Quick Sort ($\approx 700\ \mu\text{s}$) wyprzedza Shell Sort ($\approx 1\,240\ \mu\text{s}$). Wraz ze wzrostem n proporcje zmieniają się: przy $n = 100\,000$ Quick Sort ($\approx 8\,700\ \mu\text{s}$) jest wyraźnie szybszy zarówno od Bucket Sort ($\approx 11\,700\ \mu\text{s}$), jak i Shell Sort ($\approx 16\,500\ \mu\text{s}$).

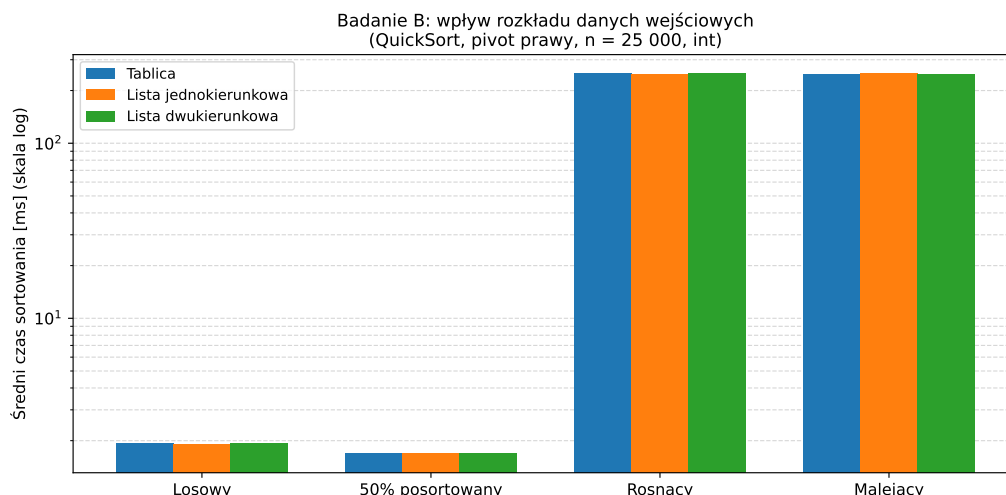
Różnica między typem struktury (tablica, lista jedno- i dwukierunkowa) jest minimalna dla wszystkich algorytmów. Wynika to z faktu, że przed sortowaniem lista jest kopiowana do tablicy pomocniczej, a po sortowaniu przepisywana z powrotem — operacja o złożoności $O(n)$, która przy dużych n jest pomijalnie mała w stosunku do czasu samego sortowania.

5.3 Badanie B — wpływ rozkładu danych wejściowych

W badaniu tym użyto Quick Sort z pivotem prawym (-p 2) dla $n = 25\,000$ elementów int na trzech strukturach liniowych. Zbadano cztery rozkłady: losowy, 50% posortowany, rosnący i malejący. Wyniki (dla tablicy) zawiera tabela 4, a wizualizację z logarytmiczną osią Y rysunek 4.

Tabela 4: Czas sortowania QuickSort (pivot prawy, tablica, $n = 25\,000$, int) dla różnych rozkładów danych.

Rozkład	Średnia [μs]	Min [μs]	Max [μs]
Losowy	1 924	1 797	2 152
50% posortowany (<code>ascending50</code>)	1 696	1 602	1 808
Rosnący (posortowany)	249 481	248 623	274 320
Malejący (odwrotny)	249 252	248 692	251 468



Rysunek 4: Średni czas sortowania QuickSort dla różnych rozkładów danych (oś Y w skali logarytmicznej).

Wyniki potwierdzają znany fakt teoretyczny: Quick Sort z pivotem z prawego krańca osiąga pesymistyczną złożoność $O(n^2)$ na danych posortowanych i odwrotnie posortowanych. Przy $n = 25\,000$ czas rośnie do $\approx 249\,000\ \mu s$ — blisko 130-krotnie więcej niż dla danych losowych.

Dla rozkładu rosnącego i malejącego czasy są praktycznie identyczne, co ma sens: w obu przypadkach pivot prawy jest zawsze minimalny albo maksymalny, więc partycja jest jednoelementowa i następuje $n - 1$ operacji podziału.

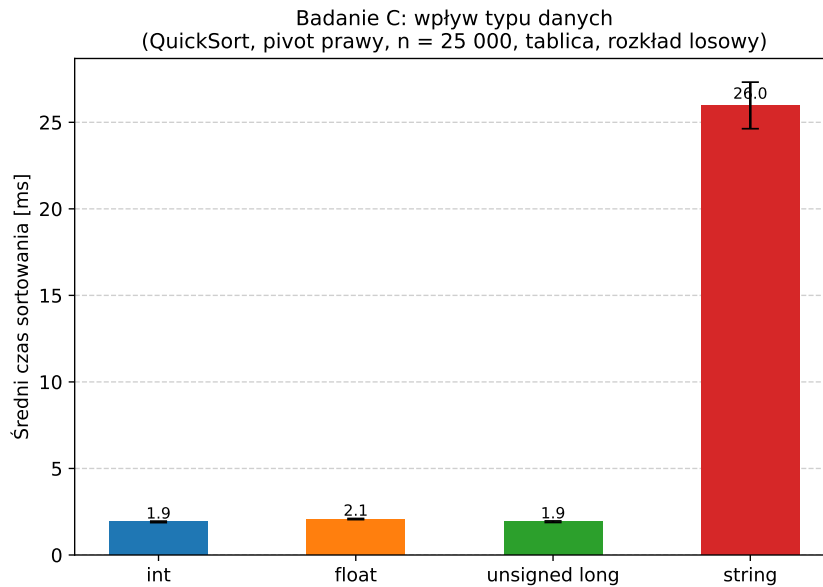
Ciekawa jest wartość dla danych 50% posortowanych: wynosi zaledwie $\approx 1\,700\ \mu s$, czyli nawet *mniej* niż dla danych czysto losowych. Wynika to stąd, że elementy częściowo ułożone w porządku wymagają mniejszej liczby przestawień podczas partycjonowania, co redukuje stały czynnik złożoności mimo identycznej asymptotyki $O(n \log n)$.

5.4 Badanie C — wpływ typu danych

Badanie C sprawdziło wpływ rozmiaru i kosztu porównania elementów na czas sortowania Quick Sort (pivot prawy, $n = 25\,000$, tablica, dane losowe). Wyniki w tabeli 5 i na rysunku 5.

Tabela 5: Czas sortowania QuickSort w zależności od typu danych ($n = 25\,000$, tablica, dane losowe).

Typ danych	Średnia [μs]	Min [μs]	Max [μs]
int	1 914	1 825	2 124
float	2 078	2 002	2 133
unsigned long	1 923	1 847	2 112
string	25 976	24 761	35 659



Rysunek 5: Średni czas sortowania QuickSort dla różnych typów danych.

Typy `int` i `unsigned long` osiągają prawie identyczny czas, co nie dziwi — oba mają taki sam rozmiar i podobny koszt porównania. `float` jest nieznacznie wolniejszy ($\approx 9\%$), co wynika z dodatkowego kosztu generowania losowych wartości zmiennoprzecinkowych (normalizacja do zakresu) oraz nieco wyższej latencji porównań `float` względem `int` na poziomie instrukcji maszynowych.

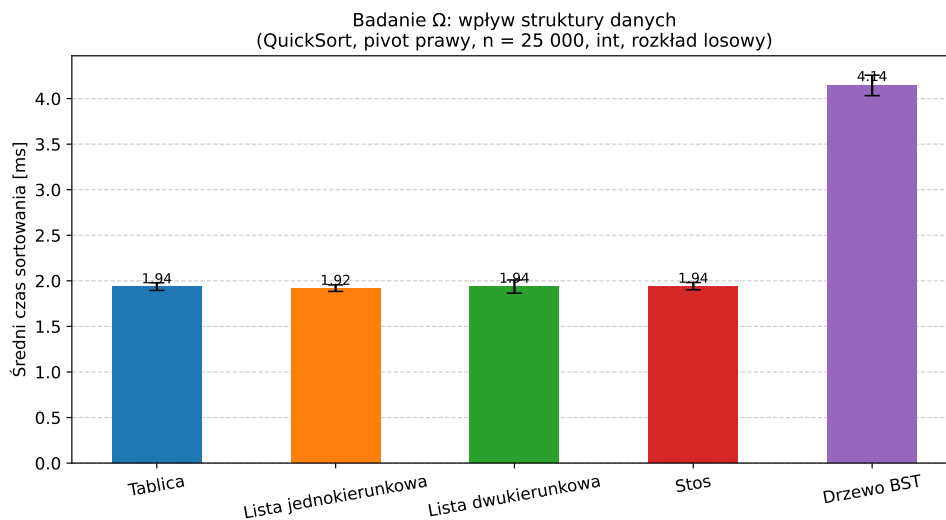
Najbardziej rzucający się w oczy jest wynik dla łańcuchów znaków: `string` jest ponad 13-krotnie wolniejszy od `int`. Każde porównanie dwóch ciągów wymaga iteracji po znakach do momentu znalezienia różnicy, a ponieważ losowe łańcuchy mają pewną minimalną długość, koszt jednego porównania jest znacznie wyższy niż porównanie liczby. Łącznie Quick Sort wykonuje $O(n \log n)$ porównań, więc ten narzut kumuluje się wyraźnie.

5.5 Badanie Ω — wpływ struktury danych

Badanie porównało pięć struktur danych: trzy liniowe (tablica, lista jedno- i dwukierunkowa), stos oraz drzewo BST, wszystkie z Quick Sort (pivot prawy, $n = 25\,000$, typ `int`, dane losowe). Wyniki zestawiono w tabeli 6 i na rysunku 6.

Tabela 6: Czas sortowania Quick Sort dla różnych struktur danych ($n = 25\,000$, int, dane losowe).

Struktura danych	Średnia [μs]	Min [μs]	Max [μs]
Tablica	1 937	1 847	2 091
Lista jednokierunkowa	1 920	1 846	2 066
Lista dwukierunkowa	1 937	1 855	2 352
Stos	1 942	1 863	2 033
Drzewo BST	4 145	3 966	4 403



Rysunek 6: Średni czas sortowania Quick Sort dla pięciu struktur danych.

Cztery struktury liniowe (tablica, dwie listy, stos) uzyskały niemal identyczne czasy w przedziale $1\,920$ – $1\,942\ \mu s$. Potwierdza to wcześniejszą obserwację z badania A: rzeczywisty czas sortowania nie zależy od typu struktury, ponieważ algorytm zawsze operuje na skopiowanej tablicy. Różnice wynikają wyłącznie z narzutu konwersji, który jest marginalny.

Drzewo BST wyróżnia się wynikiem $\approx 4\,145\ \mu s$ — ponad dwukrotnie dłużej. W tym przypadku czas mierzony obejmuje budowę drzewa przez n wstawię (każde $O(\log n)$ dla zrównoważonego drzewa, lecz bez równoważenia BST może degenerować się do $O(n)$) oraz przejście *in-order* (liniowe $O(n)$). Narzut alokacji węzłów w pamięci dynamicznej i mniejsza lokalność danych tłumaczą ten znaczący wzrost.

6 Wnioski

Przeprowadzone badania pozwoliły sformułować kilka wniosków dotyczących praktycznych własności zaimplementowanych algorytmów.

Wybór algorytmu. Dla danych losowych i typów numerycznych Quick Sort z pivotem ze środka lub prawym krańcem był najszybszy przy $n \geq 25\,000$. Bucket Sort lepiej radził sobie dla mniejszych zbiorów ($n \leq 10\,000$), natomiast Shell Sort konsekwentnie plasował się na trzecim miejscu — jego złożoność jest wyższa niż $O(n \log n)$ Quick Sorta (formuła Shella ma pesymistyczne $O(n^2)$, formuła Knutha — $O(n^{3/2})$).

Wybór pivota. Strategia losowa jest ok. 44% wolniejsza od deterministycznych, co wynika z narzutu wywołania generatora liczb pseudolosowych przy każdym podziale. Dodatkowo charakteryzuje się wyższą wariancją czasu (odch. std. $\approx 250 \mu\text{s}$ wobec $\approx 30\text{--}50 \mu\text{s}$ dla pozostałych strategii). Na danych losowych pivot prawy lub środkowy jest szybszy i bardziej przewidywalny.

Rozkład danych. Quick Sort z pivotem prawym jest ekstremalnie wrażliwy na dane posortowane — czas rośnie z $\approx 2 \text{ ms}$ do $\approx 250 \text{ ms}$, czyli wzrost prawie 130-krotny. W zastosowaniach produkcyjnych wybór pivota ze środka lub podejście *median-of-three* jest niezbędny do zachowania efektywności.

Typ danych. Typy całkowite i zmiennoprzecinkowe są porównywalne pod względem czasu sortowania. Łańcuchy znaków powodują kilkunastokrotny wzrost czasu ze względu na droższe porównania.

Struktura danych. Dla algorytmów operujących wewnątrz na tablicy wybór struktury liniowej (tablica, lista, stos) nie ma praktycznego wpływu na wydajność sortowania. Drzewo BST jest wyraźnie wolniejsze z powodu rozłożonej alokacji pamięci i większej liczby chybień cache.

Sekwencja Knutha. Shell Sort z sekwencją Knutha ($3k + 1$) okazał się systematycznie szybszy ($\approx 15\%$) od oryginalnej formuły Shella ($n/2$). Dla większych zbiorów różnica ta może być jeszcze bardziej widoczna.